

AI-Enhanced Music Mashup Studio V1.0.1

Complete Design Document, Visual Design & Component Mapping

Table of Contents

- 1. [\[Overall Layout Philosophy\]](#)
- 2. [\[Screen Layout with Component Names\]](#)
- 3. [\[Modals\]](#)
- 4. [\[Component Architecture\]](#)
- 5. [\[Service / Utility Layers\]](#)
- 6. [\[Data Persistence Model\]](#)
- 7. [\[Design Decisions\]](#)

Overall Layout Philosophy

A three-panel application layout designed for a professional audio workflow:

- **Global Header:** Persistent project management and master transport controls.
- **Left Sidebar (Library Panel):** Collapsible sidebar managing local MP3 uploads and a browseable Spotify catalog.
- **Main Workspace:** Central scrolling area containing up to 5 vertically stacked tracks with expandable settings.
- **Right Sidebar (AI Panel):** Collapsible conversational AI chat interface for mixing guidance.

Each track in the Main Workspace features:

- In-line audio controls (play/pause, mute, volume).
- Expandable settings drawer (EQ, Pitch, Speed, Effects, Segments).
- Reorderable via drag-and-drop.

Screen Layout with Component Names

```
```text
+-----+
| <Header> |
| |
| [Logo] [ProjectNameEditor] | ► [MasterStatus] [MasterBPM] [VolToggle] |
+-----+```
```

```

[Save] [Load] [Export] [User] |
+-----+
+-----+
| <LibraryPanel> (Left Sidebar) | <MainWorkspace> (Center) | | | | | |
| <AIPanel> (Right Sidebar) | | |
| | |
| | |
| [Upload MP3] | <TrackStack> |
| <ChatHistoryModalToggle> | | |
| | |
| _____ | | 🤖 AI Studio Assistant |
| | |
| <LocalFilesGroup> | | <TrackCard> | |
| | |
| • "Song 1" [Delete] | | <TrackHeader> | |
| <ChatWindow> | | |
| | | [Drag] "Title" - Artist [X] | |
| | |
| <SpotifyCatalogGroup> | | [BPM] [Key] | |
| [System Note: Context...] | | |
| • Playlist 1 [ModalTrigger] | | </TrackHeader> | |
| | |
| • Playlist 2 | | | |
| <UserMessage> | | |
| | | <WaveformDisplay> | |
| <AIMessage> | | |
| | | [|||] [Mute] [Vol-0—] | |
| | |
| | | [Waveform=====] |
| | |
| | | </WaveformDisplay> | |
| | |
| | | | |
| <ChatInputArea> | | |
| | | <TrackSettings> (Expanded) | |
| [Ask for track tips...] [↑] | | |
| | | > Basic Controls | |
| | |
| | | > Audio Adjustments (Pitch/Spd) |
+-----+
| | | > Equalizer (Lo/Mid/Hi) |
| | | > Audio Effects |
| | |

```

```
|
| | [+ Add New Track (x/5)]
+-----+-----+-----+-----+
|
```

## Modals

Component	Trigger
<PlaylistModal>	Clicking a playlist in the Spotify Catalog to view/add tracks
<AuthScreen>	First load or when unauthenticated
<ProfileModal>	User profile picture — contains General and Controls settings
<ChatHistoryModal>	History icon in the AI Sidebar
Load Project modal	"Load" button in Header; lists and restores saved projects

## Component Architecture

### 1. App Level

```
`App.js`
```

**Purpose:** Root component

**Responsibilities:**

- Wraps the app in `MixProvider` (appContext.js) and `SpotifyContext`.
- Renders `<AuthScreen />` when the user is unauthenticated.
- Coordinates the three-panel layout.

**Child Components:** <Header>, <LibraryPanel>, <MainWorkspace>, <AIPanel>

### 2. Header

```
`Header.js`
```

**Responsibilities:**

- Display and edit workspace Project Name.
- Master transport controls (Play/Pause, Stop, Master BPM, Global Zoom).
- Save (writes to Firestore), Load (opens project list modal), Export (offline WAV render), Mix Preview.
- Undo/Redo (50-step history from appContext.js).
- User profile access via avatar dropdown.

### 3. Left Sidebar

``LibraryPanel.js``

#### Responsibilities:

- Upload MP3 functionality (reads ID3 tags, submits to AudD for fingerprinting, falls back to Claude filename parsing).
- Manages ``LocalFilesGroup`` displaying the user's uploaded tracks from Firebase Storage.
- Manages ``SpotifyCatalogGroup`` showcasing available Spotify playlists as a browsable catalog.
- Toggle visibility of the left sidebar.

### 4. Main Workspace

``MainWorkspace.js``

- Manages the drag-and-drop drop zone logic and GapZone components.
- Surfaces track limit error and storage quota error banners from appContext.js.
- Renders ``<TrackCard>`` for each track in the workspace (max 5 distinct base songs).

``TrackCard.js``

All per-track UI is consolidated in this single component. Sub-sections are not separate files:

- **Track header:** Editable title, artist, BPM, key display, drag handle, delete/duplicate controls.
- **Waveform:** WaveSurfer.js instance; visualizes audio, scrub position, segment markers.
- **Basic Controls:** Volume, fade-in, fade-out, offset.
- **Audio Adjustments:** Pitch ( $\pm$ semitones) and Speed (ratio) with reset buttons and quality warning.
- **Equalizer:** Lo/Mid/Hi gain sliders per track.
- **Audio Effects:** Reverb, delay, compressor, HPF, LPF, panner, volume; each independently toggled.
- **Segments:** Configurable regions with start/end, pitch, speed, fades per segment.

## 5. AI Panel

``AIPanel.js``

All AI chat UI is consolidated in this single component. Sub-sections are not separate files:

- **Chat sessions:** Up to 5 named sessions, create/rename/delete.
- **Message rendering:** User messages and Claude replies with MarkdownMessage renderer.
- **Context timestamp:** Each AI reply shows "Context at HH:MM" indicating when workspace state was captured.
- **Bias disclosure:** Persistent banner above all messages.
- **Input area:** Text input and send button.

Claude is accessed via the ``/api/aiChat`` proxy endpoint. There is no separate AI scoring or recommendation service, all AI interaction is conversational.

## Service / Utility Layers

``firebase.js`` + ``useFirebaseAuth``

- Google OAuth and email/password authentication.
- Profile operations (updateDisplayName, updateProfilePhoto, updateUserEmail).

``FirebaseService.js``

- Firebase Storage: upload, download URL retrieval, delete MP3 files.
- Firestore: track metadata CRUD (upload/delete records).
- Project persistence: ``saveProject``, ``loadProject``, ``getUserProjects``, ``deleteProject``.

``appContext.js`` (MixProvider / useMix)

- All workspace state: tracks array, universallsPlaying, undo/redo history (50 steps).
- UID-scoped localStorage auto-save with 500 ms debounce; amber banner shown on quota failure.
- Exports: ``handleAddTrack``, ``handleDeleteTrack``, ``handleDuplicateTrack``, ``handleMoveTrack``, ``handleUpdateTrack``, ``handleOverwriteTracks``, ``handleClearAllTracks``, ``trackLimitError``, ``storageError``.

``spotifyApi.js`` / SpotifyService

- Spotify OAuth flow (PKCE).

- Playlist fetch and catalog track search.
- All Spotify data is read-only, used as a catalog.

#### ``AudioEngine.js` + `useAudioEngine.js``

- Multi-track Web Audio API playback coordination.
- SoundTouch integration for real-time pitch/speed via ScriptProcessorNode.
- Per-track EQ (BiquadFilterNode), effects chain (reverb, delay, compressor, HPF, LPF, panner), volume.
- Offline rendering (``renderOffline``) for WAV export.
- ``useAudioEngine.js`` is the React hook that bridges TrackCard state changes to AudioEngine.

#### ``essentiaAnalyzer.js``

- Client-side BPM, key, and beat position analysis via Essentia.js Web Worker.
- 30-second timeout, rejects with "Essentia analysis timed out after 30s" if the worker never responds.

#### ``trackConfig.js``

- Defines ``EFFECT_CONFIGS`` (all effects, their parameters, valid ranges, labels).
- Exports ``buildEffectsCapabilities()``; generates the AI system prompt effects block at runtime from ``EFFECT_CONFIGS`` so the AI always has accurate knowledge of available controls.

#### ``helpers.js``

- ID3 tag parsing, AudD confirm match, Spotify query builder, keybind utilities.

#### ``useSettings.js``

- Keybind and default audio value persistence per user (localStorage).

## Data Persistence Model

Audio files are stored in Firebase Storage. Track metadata and project saves are in Firestore. The active workspace auto-saves to browser ``localStorage`` (UID-scoped) on every change.

## Track Object (Internal State)

```
```json
{
  "id": "unique-uuid-or-timestamp",
  "title": "Track Name",
  "artistName": "Artist",
  "albumArt": "https://...",
  "spotifyId": "spotify:track:...",
  "audioUrl": "https://firebasestorage...",
  "bpm": 128,
  "trackKey": "C major",
  "beatPositions": [],
  "initialSegments": [],
  "initialVolume": 1.0,
  "initialPitch": 0,
  "initialSpeed": 1.0,
  "initialFadeIn": 0,
  "initialFadeOut": 0,
  "initialZoom": 0,
  "offsetSec": 0,
  "isMissing": false,
  "initiallyExpanded": false
}
```
```

Note: EQ, effects, and per-segment audio parameters are managed by AudioEngine state and not stored on the track object directly. Only initial values and segment definitions are serialized to localStorage and Firestore.

## Design Decisions

| Decision                          | Rationale                                                                                                     |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------|
| Three-Panel Density               | Keeps the user in a professional, hardware-style flow rather than simplified linear views.                    |
| TrackCard as monolithic component | All per-track UI consolidated in one file; avoids prop-drilling overhead across deeply nested sub-components. |
| Client-Side Analysis              | Local analysis via Essentia.js prioritizes low server cost and zero network dependency for                    |

|                         |                                                                                                                                                                                    |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         | BPM/key.                                                                                                                                                                           |
| Spotify as catalog only | Spotify acts as a pre-filtered catalog of known playlists rather than a raw search field or scored recommendation engine, guiding users faster without AI bias in track discovery. |
| Conversational AI       | Non-destructive, advisory Claude interface rather than an auto-mixing engine preserves human agency. No structured recommendation objects; all AI output is plain text in chat.    |
| UID-scoped localStorage | Workspace auto-saves per user so multiple accounts on the same browser don't overwrite each other.                                                                                 |